```
query SecondFloorTemps {
  rooms(filter: {floor: "2"}) {
    name
    sensor {
      latestMeasurement {
        temperature
      }
    }
  }
}
```

In this approach the client gets exactly the temperatures needed, traversing the relationships between rooms, sensor, and latestMeasurement in a single, highly efficient round trip.

For digital twins built on interconnected data, GraphQL offers the advantage of allowing clients to efficiently query the entire graph, minimizing latency and maximizing bandwidth usage.

5.6.3 The power of a GraphQL schema

The power of using GraphQL lies in defining a schema that represents the specific domain which in this case is my home. This schema acts as a contract, outlining all the data that can be queried and how different data points relate to each other. For my home digital twin, I've based my schema on the RealEstateCore ontology. This provides several key advantages:

- *Standardization* - by adopting an existing, well-defined ontology, I am not creating a custom, isolated data model. RealEstateCore provides a standardized vocabulary for concepts like buildings, floors, rooms, and sensors.
- *Interoperability* - this standardization ensures my digital twin can interoperate with other systems that also use the RealEstateCore ontology as they can speak the same language as my digital twin, facilitating data exchange and integration.
- *Context and meaning* - the ontology provides the semantic foundation that turns raw data into meaningful information. A simple temperature reading of 25° becomes a data point with context—it's the temperature